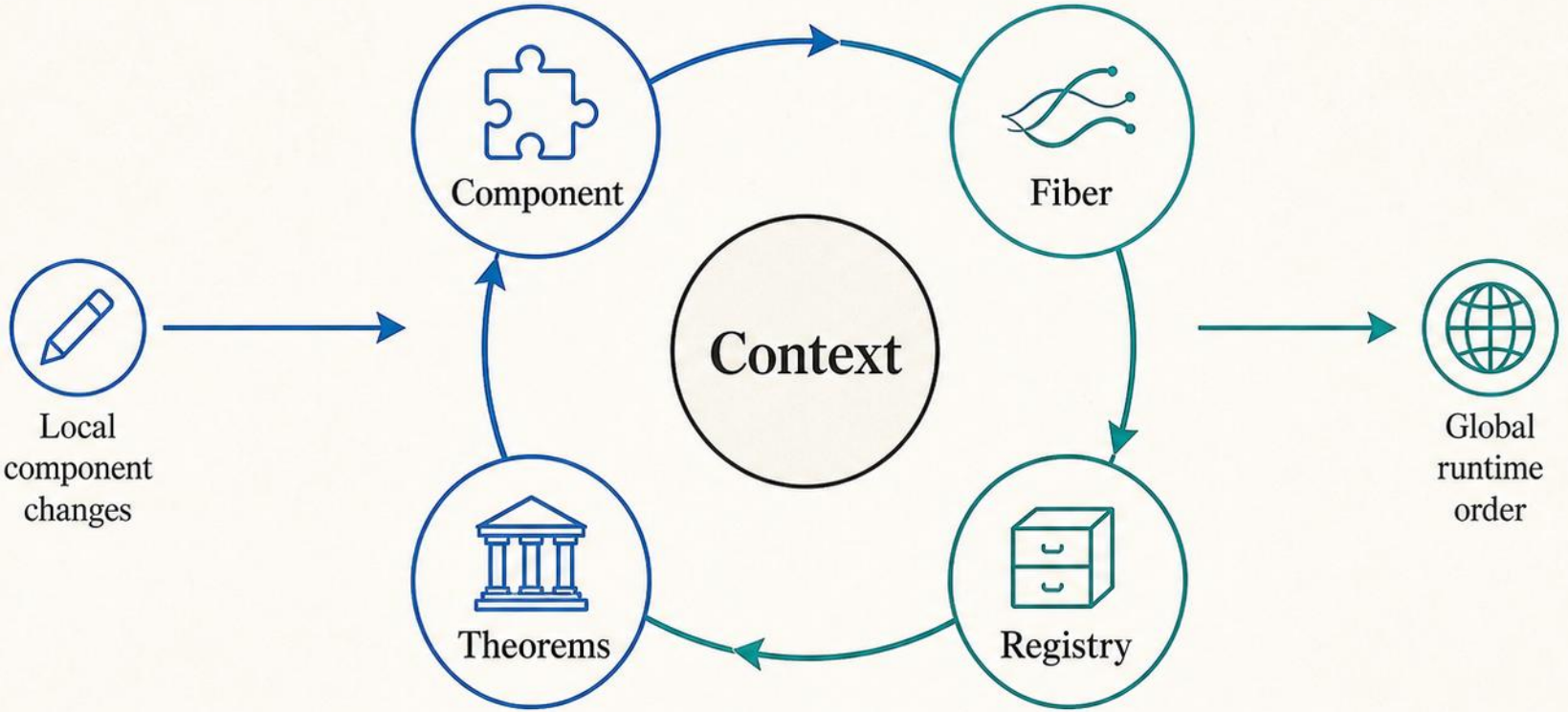
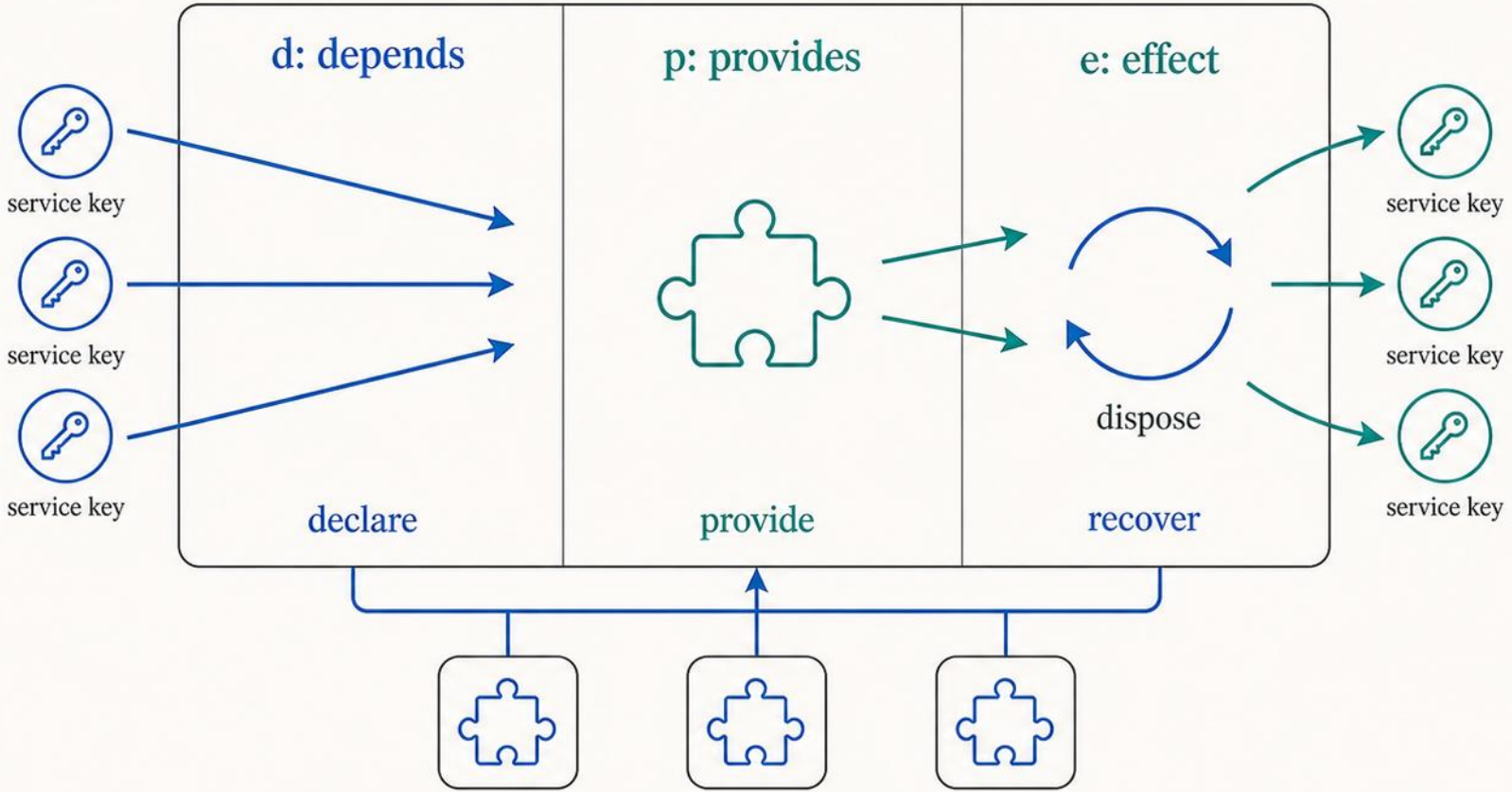
组件演算与元理论

A Programming Paradigm for Spatiotemporal Composability 解读 · Part 2



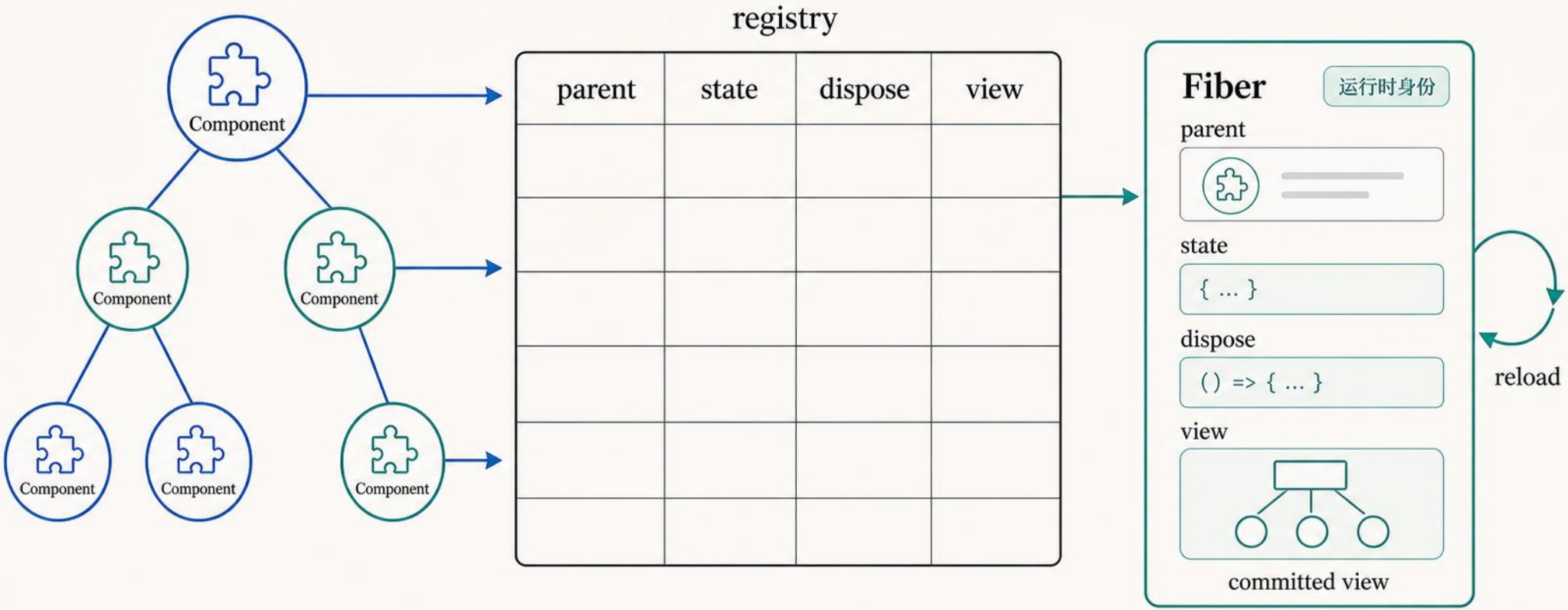
Component = d / p / e

一个组件同时声明依赖、提供能力、产生可恢复效果



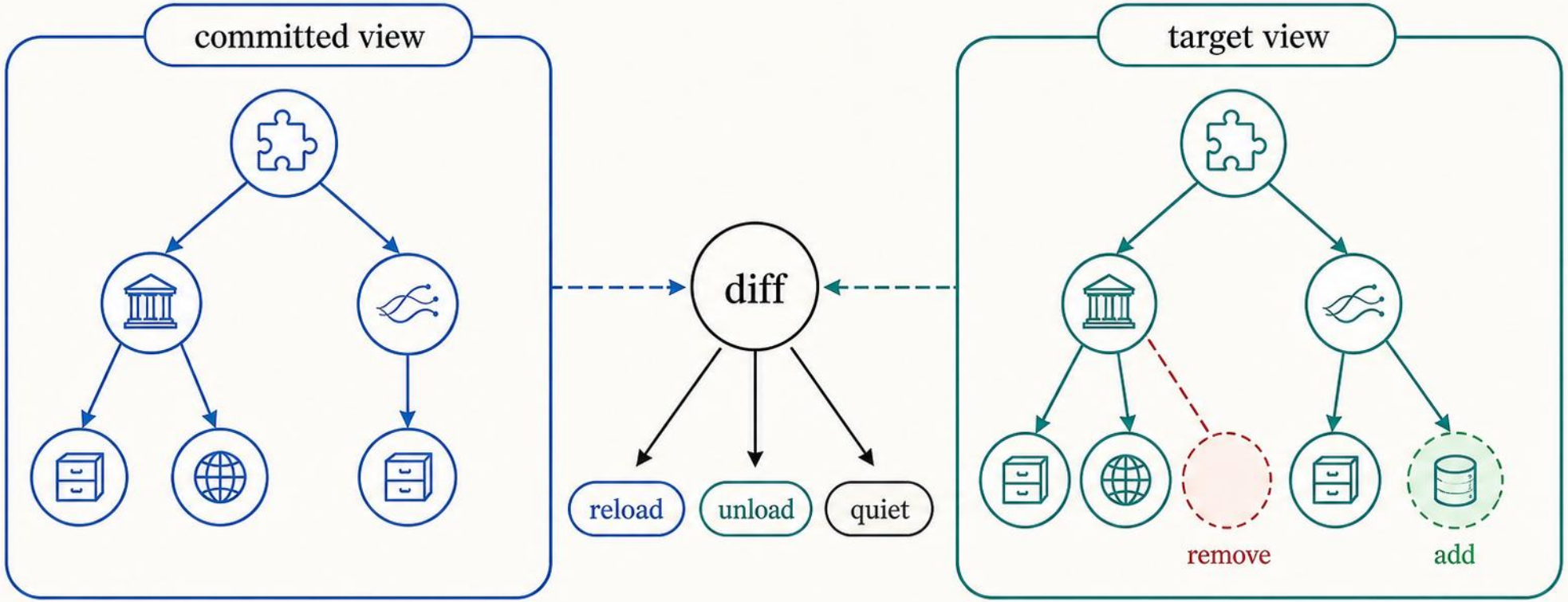
Fiber：组件的运行时身份

fiber 记录父子关系、状态、清理函数和已提交视图



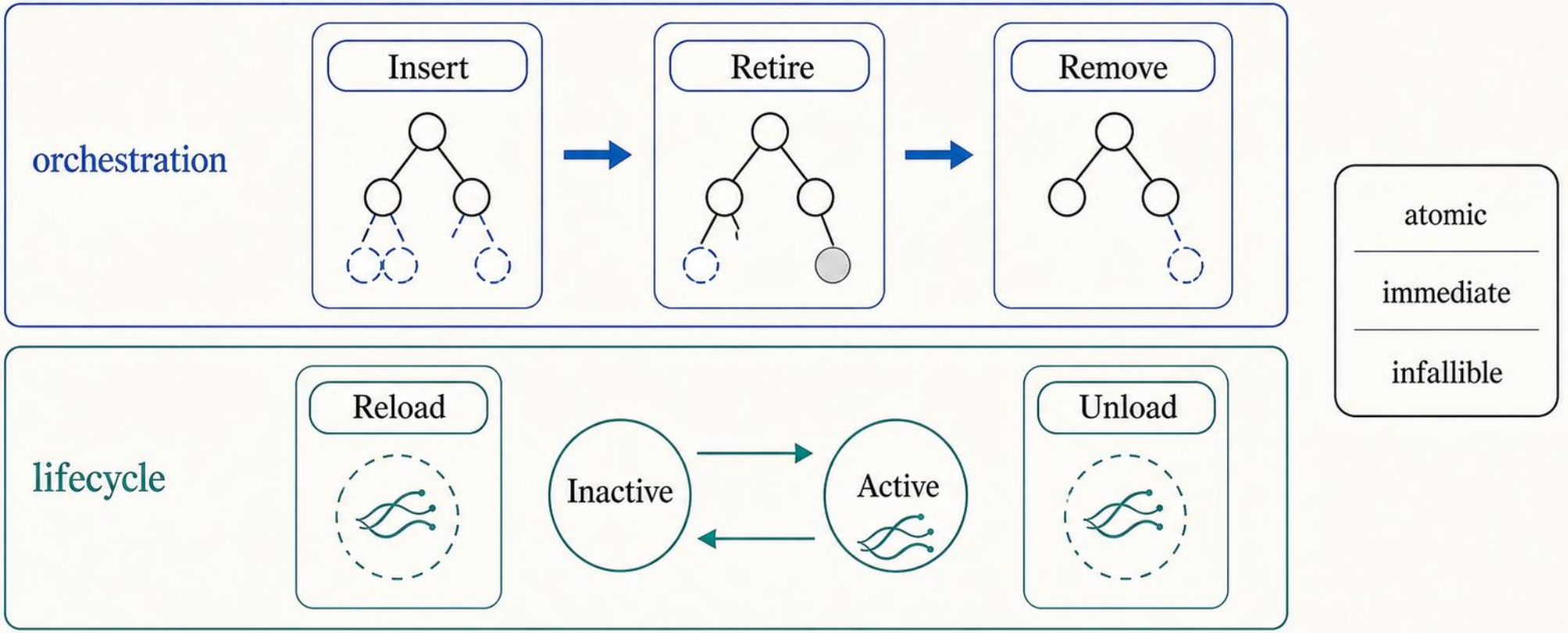
Target View：把变化变成动作

运行时比较已提交视图与目标视图，再决定 reload / unload / quiet



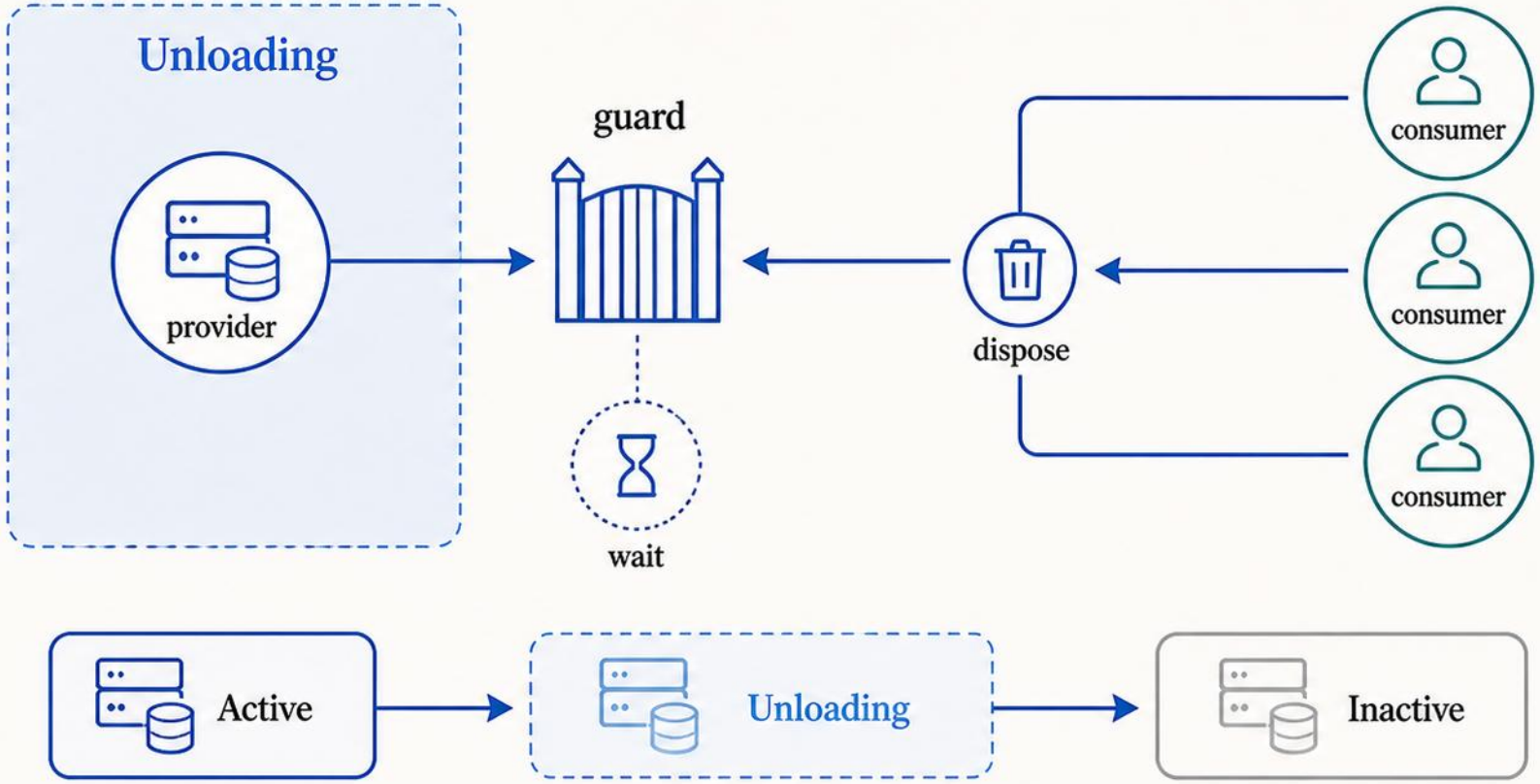
Base Calculus：两层动作

编排动作改变树，生命周期动作恢复或激活组件



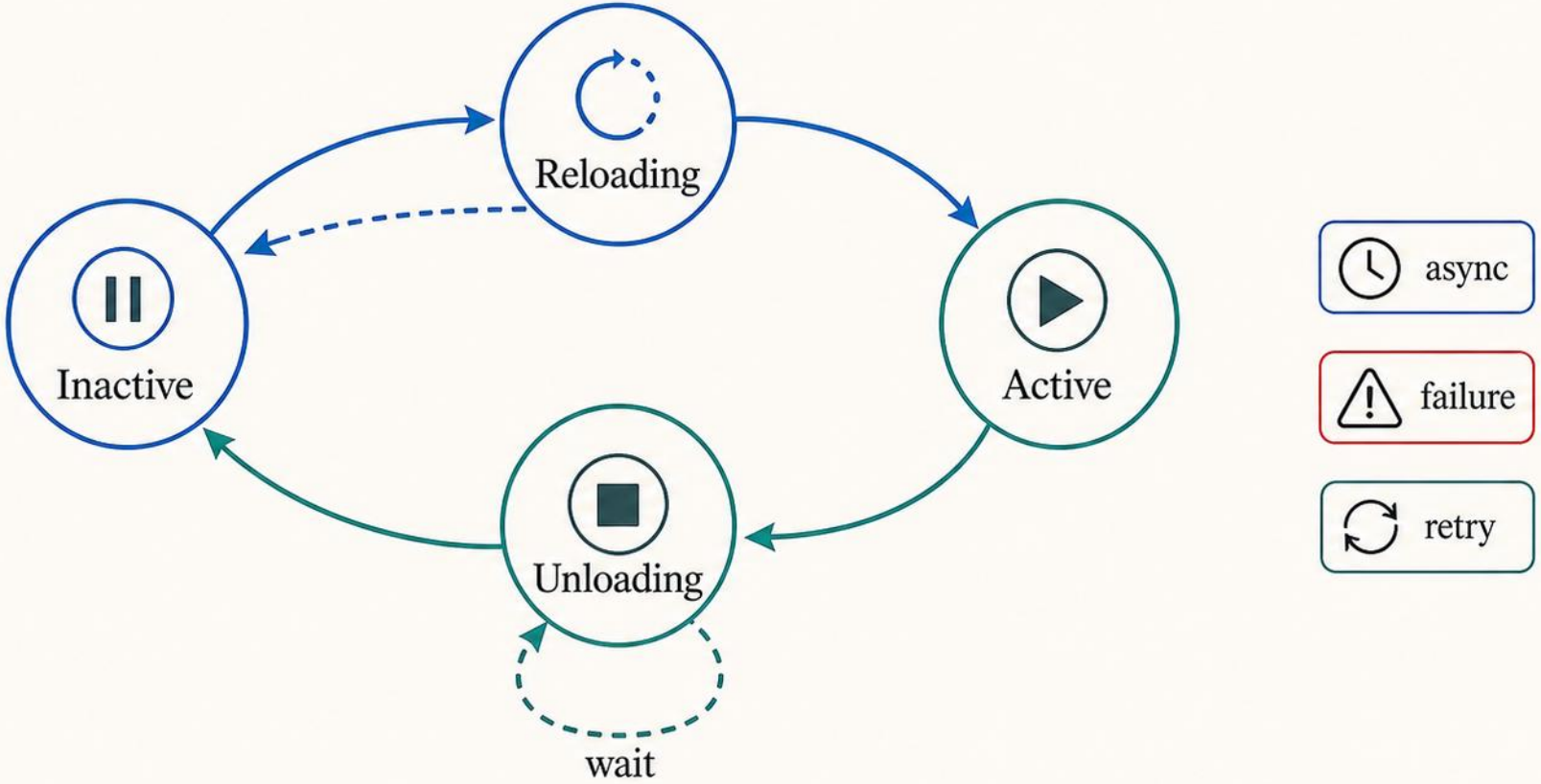
为什么需要 Unloading

提供者先进入卸载区，让依赖者有序撤离



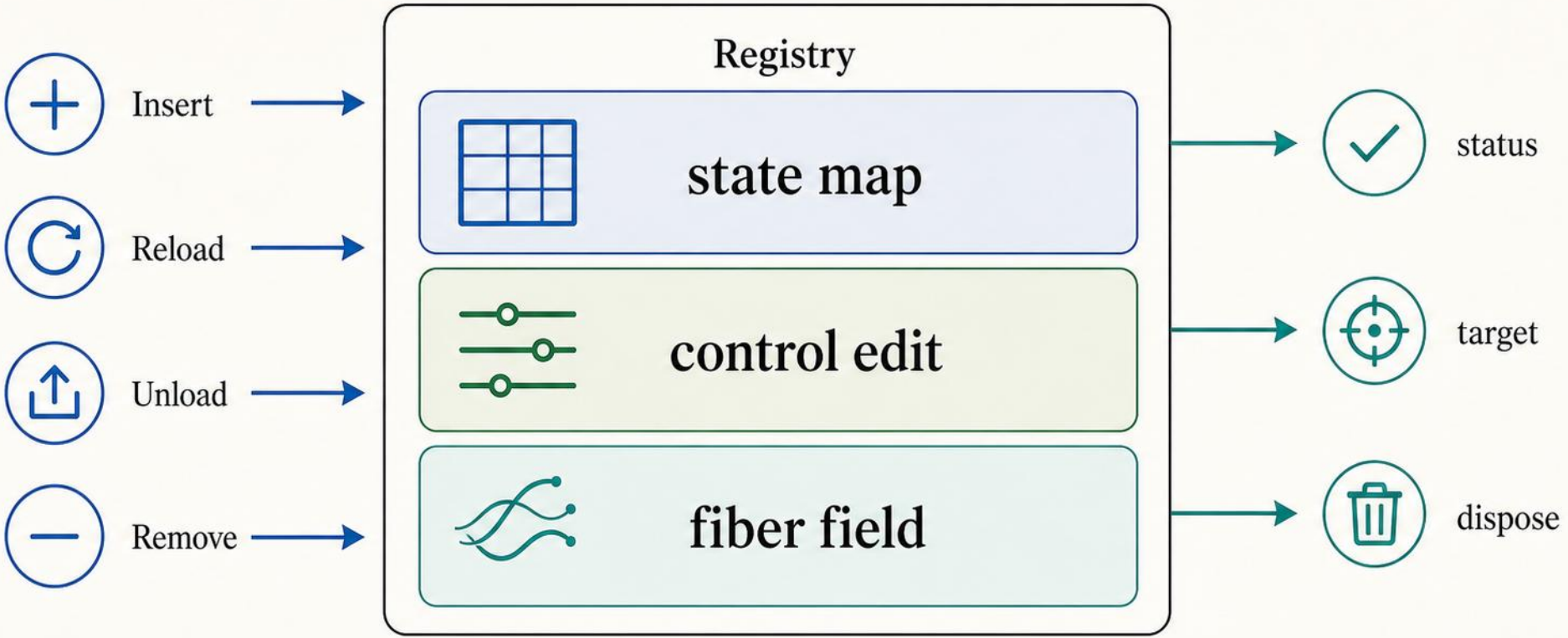
真实运行时：四个状态

异步 reload / unload 需要显式中间态承接失败与重试



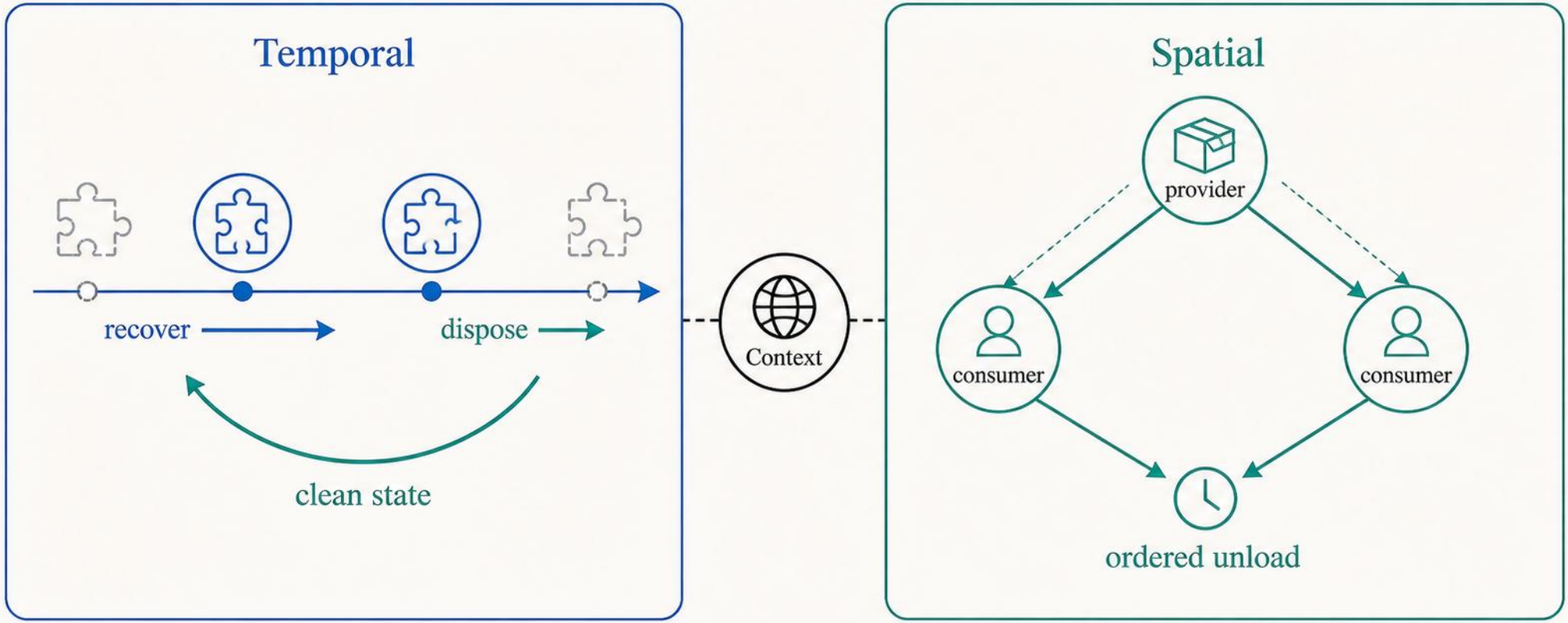
规则 = Registry writes

Table 1 simplified



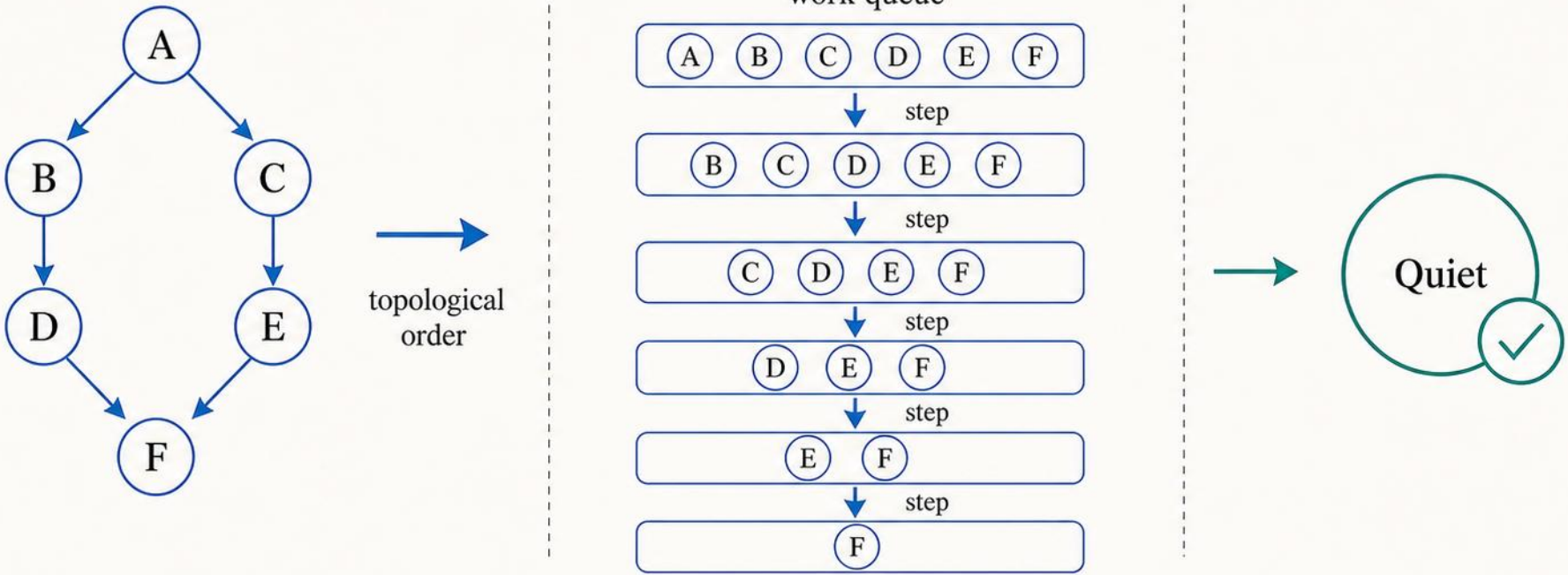
两个保证

temporal recovery + spatial ordering



Progress：不会卡死

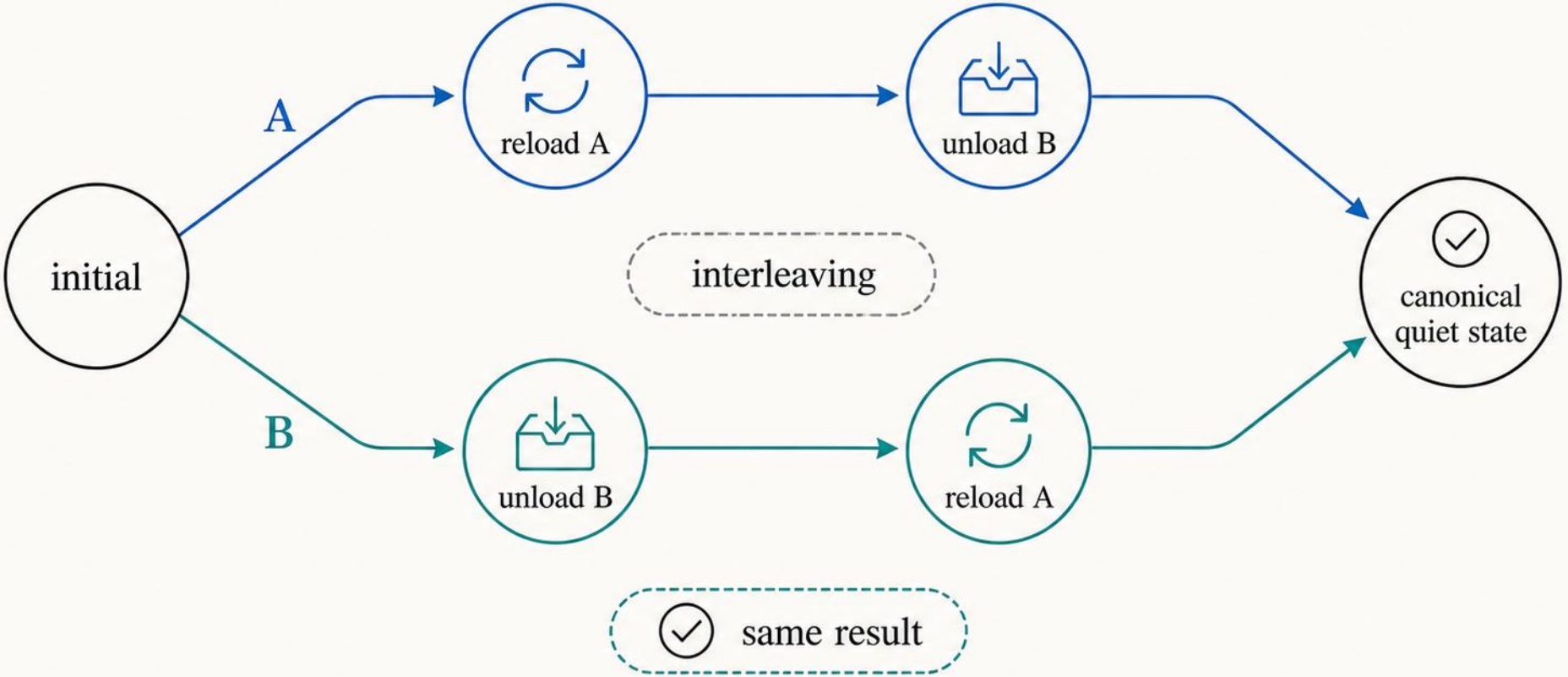
依赖图无环时，每一步都减少未完成工作



Acyclic | No deadlock | Termination

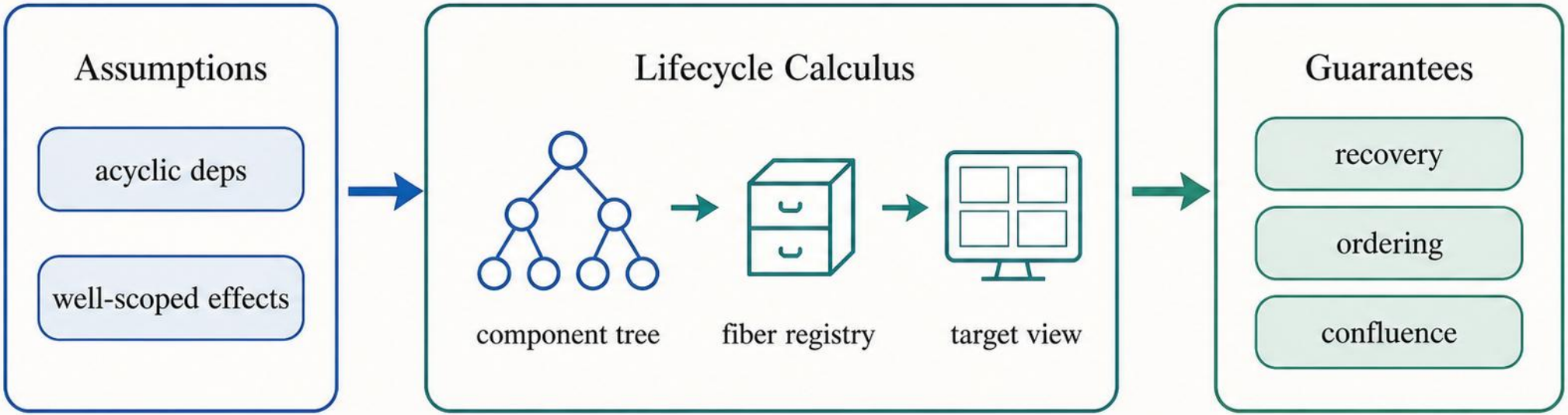
Confluence：执行顺序不应改变结果

合法交错可以不同，但安静后的 canonical state 一致



Part 2 小结

局部组件声明 + 全局生命周期调度 = 可证明的组合



Part 3: Cordis